4G3 Computational Neuroscience — Assignment 1

Blind Grading Number: **8415L**

AI Disclosure. This report was drafted by hand and refined with AI assistance. GitHub Copilot was used for autocomplete during writing. Claude Code was used for coding assistance and report drafting. Prompts are listed in Appendix B. All simulations were verified against the handout; all results were inspected and interpreted by the author.

Question 1

The general function-approximation TD update is:

$$\theta_k \leftarrow \theta_k + \varepsilon\left[r_t + \gamma\hat{V}^\pi(s_{t+1}) - \hat{V}^\pi(s_t)\right]\frac{\partial f(s_t, \boldsymbol{\theta})}{\partial \theta_k} \tag{1}$$

To recover tabular TD, let $\boldsymbol{\theta}$ have one entry per state and define $f(s, \boldsymbol{\theta}) = \sum_j \theta_j \mathbb{1}(s = j)$. Then $\frac{\partial f}{\partial \theta_k} = \mathbb{1}(s_t = k)$, giving:

$$\theta_k \leftarrow \theta_k + \varepsilon\left[r_t + \gamma\hat{V}^\pi(s_{t+1}) - \hat{V}^\pi(s_t)\right]\mathbb{1}(s_t = k) \tag{2}$$

Only the parameter for the current state is updated, which is exactly lookup-table TD(0).

Question 2

For the linear approximator $f(s, \boldsymbol{w}) = \sum_k w_k \varphi_k(s)$, the partial derivative is simply $\frac{\partial f}{\partial w_k} = \varphi_k(s_t)$. Substituting into Equation 1:

$$w_k \leftarrow w_k + \varepsilon\underbrace{\left[r_t + \gamma\hat{V}^\pi(s_{t+1}) - \hat{V}^\pi(s_t)\right]}_{\delta_t}\varphi_k(s_t) \tag{3}$$

Each weight update is the product of two factors: a scalar TD error δ_t (shared across all k) and the feature activation $\varphi_k(s_t)$. Features that are active in the current state absorb the prediction error; inactive features are unchanged. This enables generalisation: states sharing features transfer learned value.

Question 3

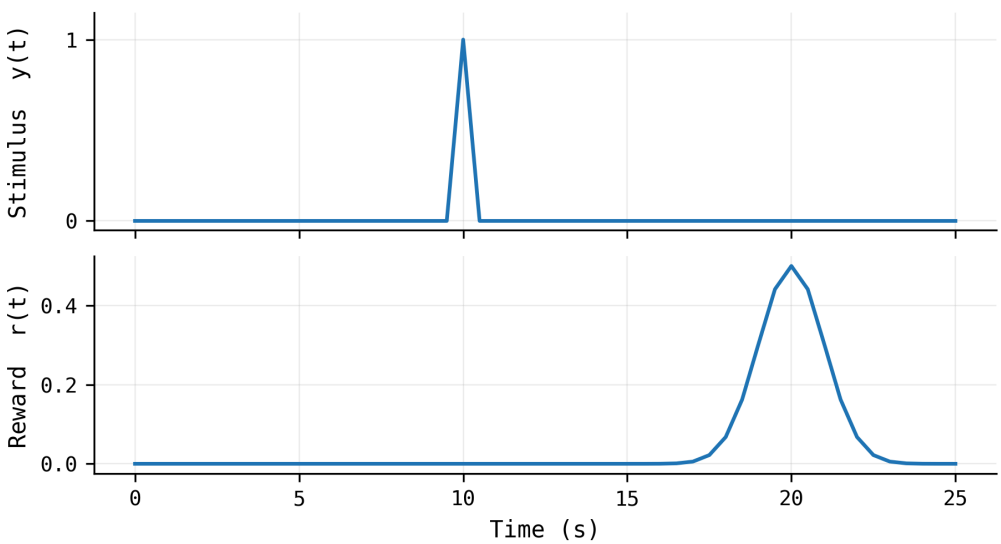

Figure 1: Stimulus and reward as functions of time.

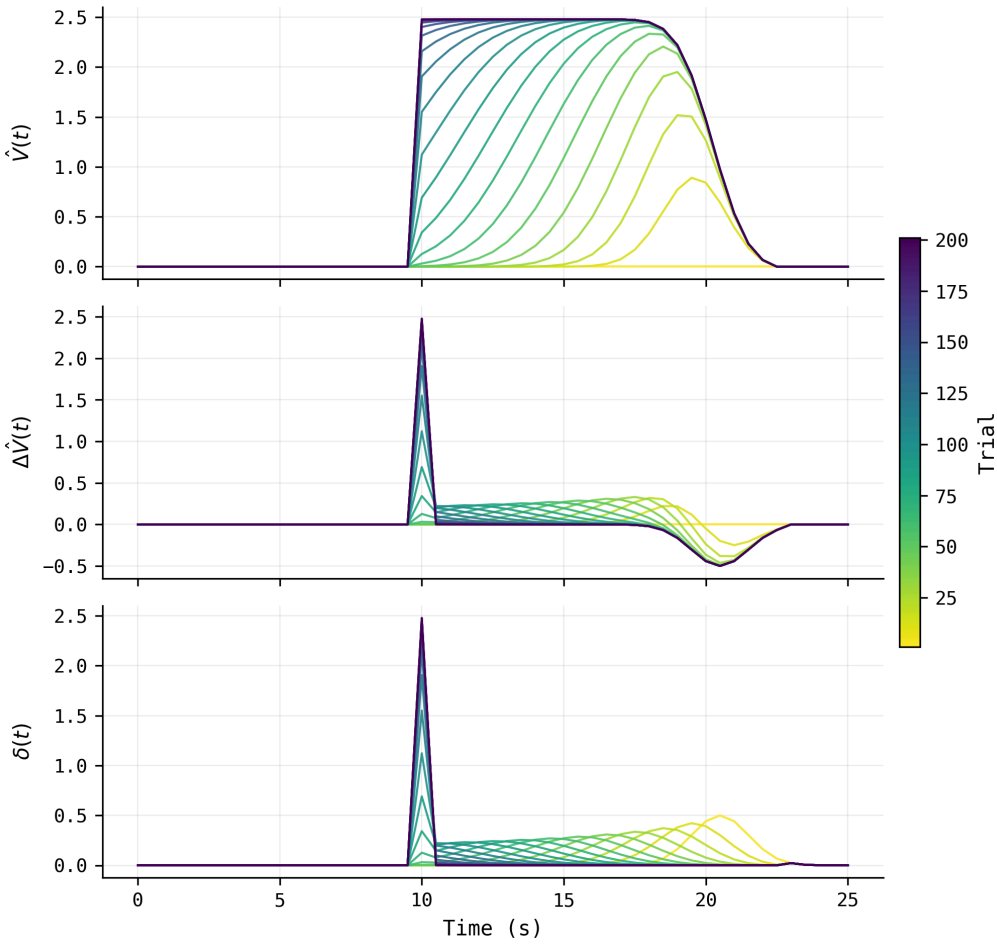

Figure 2: $\hat{V}(t)$, $\Delta\hat{V}(t)$, and $\delta(t)$ across 21 sampled trials.

(c) Early in learning $\hat{V}(t)$ is near zero everywhere. The TD error $\delta(t)$ is large and positive at reward time because the reward is unexpected. As learning progresses, $\hat{V}(t)$ develops a ramp that rises from stimulus onset to reward time: the agent learns that the stimulus predicts reward. The temporal difference $\Delta\hat{V}(t)$ shows where the value function is increasing most steeply, producing a positive bump that migrates backward from the reward toward the stimulus across trials. The TD error $\delta(t)$ correspondingly shifts: the positive peak at reward time shrinks (reward becomes expected) while a new peak emerges at stimulus onset (the stimulus itself becomes a surprising predictor of future value).

(d) The TD error $\delta(t)$ corresponds to dopamine cell activity. This agrees with experimental observations: initially dopamine fires at reward delivery; after conditioning, the dopamine response shifts to stimulus onset while the reward-time response vanishes. In Figure 2, we see exactly this progression — the $\delta(t)$ peak migrates from $t = 20$ s to $t = 10$ s across trials.

Question 4

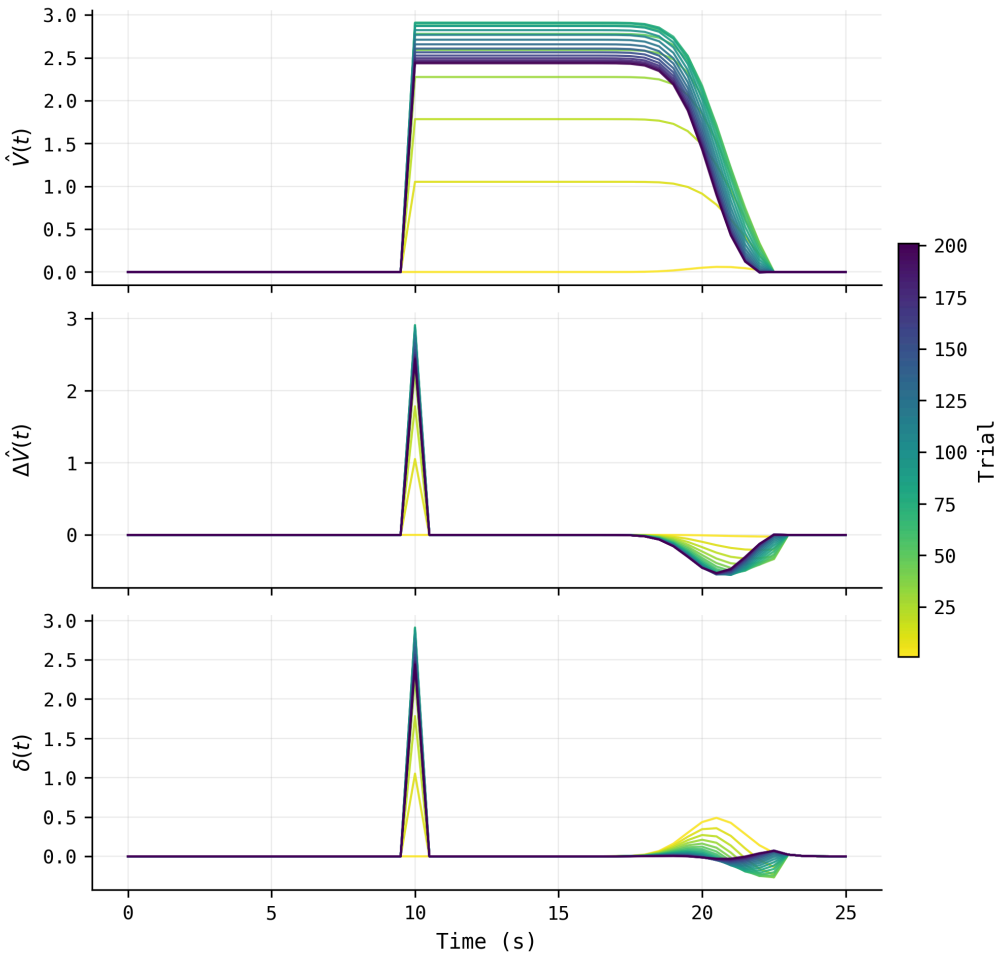

Figure 3: Boxcar representation: $\hat{V}(t)$, $\Delta\hat{V}(t)$, and $\delta(t)$ across 21 sampled trials ($\varepsilon = 0.01$).

(b) Compared with the tapped delay line (Figure 2), the boxcar representation produces a smoother value function because each feature integrates stimulus presence over a range of past time steps, providing temporal generalisation. In the first trial, $\hat{V}(t)$ and $\delta(t)$ are identical to the tapped case (weights start at zero). By the last trial, $\hat{V}(t)$ has converged to a smooth ramp rather than a sharp step, and $\delta(t)$ shows a cleaner shift with less residual noise. During intermediate trials the convergence is faster in terms of the qualitative shape because boxcar features share information across time lags.

(c) The $\delta(t)$ panel in Figure 3 shows the dopamine "shift": the reward-time peak progressively disappears and is replaced by a stimulus-onset peak. This is smoother than in Q3 because the boxcar features allow each weight update to influence predictions at multiple time steps, so the shift happens more gradually and completely within the 201 trials.

(d) The learning rate must be reduced ($\varepsilon = 0.01$ vs 0.2) because boxcar features are much larger in magnitude (they sum up to $\tau + 1$ stimulus values rather than being 0 or 1). The effective step size is $\varepsilon \cdot \varphi_k(s_t)$; with boxcar features φ_k can be as large as the memory span, so a large ε causes weight oscillations and divergence. Reducing ε compensates for the increased feature magnitude and stabilises learning.

Question 5

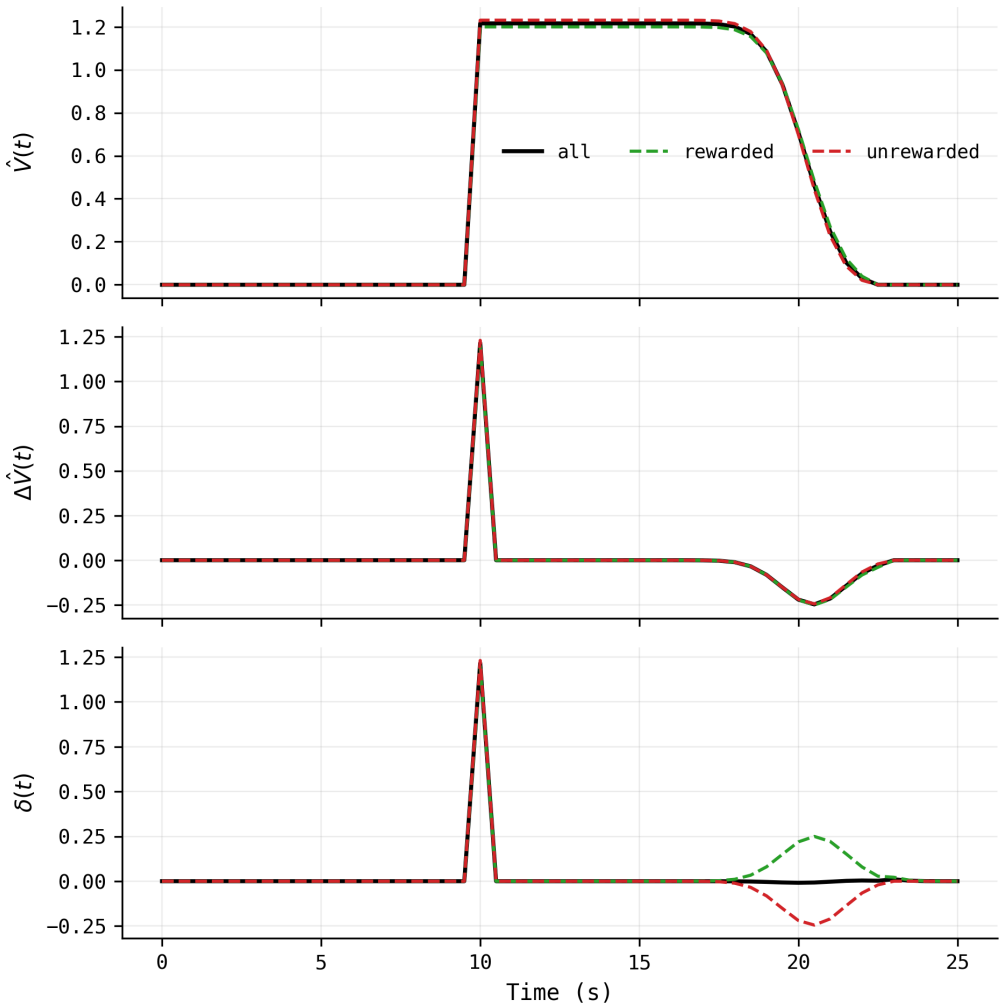

Figure 4: Average $\hat{V}(t)$, $\Delta\hat{V}(t)$, and $\delta(t)$ over the last 100 trials, split by rewarded, unrewarded, and all trials ($p = 0.5$).

(b) The value $\hat{V}(t)$ and the temporal difference $\Delta\hat{V}(t)$ are *identical* across rewarded, unrewarded, and all trials because they depend only on the weights $\boldsymbol{w}$ and the stimulus (which is the same on every trial). The weights are shared across trial types, so $\hat{V}$ does not distinguish whether reward will be delivered. Only $\delta(t)$ differs: at reward time ($t \approx 20$ s), rewarded trials show a positive δ (reward exceeds expectation under $p = 0.5$) while unrewarded trials show a negative δ (omitted reward falls below expectation). At stimulus onset all three averages coincide because the stimulus is identical.

(c) The TD error $\delta(t)$ represents dopamine neuron activity. Key features matched by this variable: (1) a positive response at stimulus onset (conditioned stimulus predicts value), (2) a positive burst on rewarded trials at reward time (reward exceeds the $p = 0.5$ expectation), and (3) a negative dip on unrewarded trials at reward time (omitted reward is worse than expected). This matches the classic pattern of dopamine responses in partial reinforcement.

(d) Interpreting dopamine as "surprise" or "salience" would predict a *positive* dopamine response whenever something unexpected happens, regardless of whether it is good or bad. However, on unrewarded trials the omission of expected reward is surprising yet produces a *negative* δ. The TD error is a *signed* prediction error: it is positive when outcomes exceed expectations and negative when they fall short. A pure surprise/salience account does not explain negative dopamine dips.

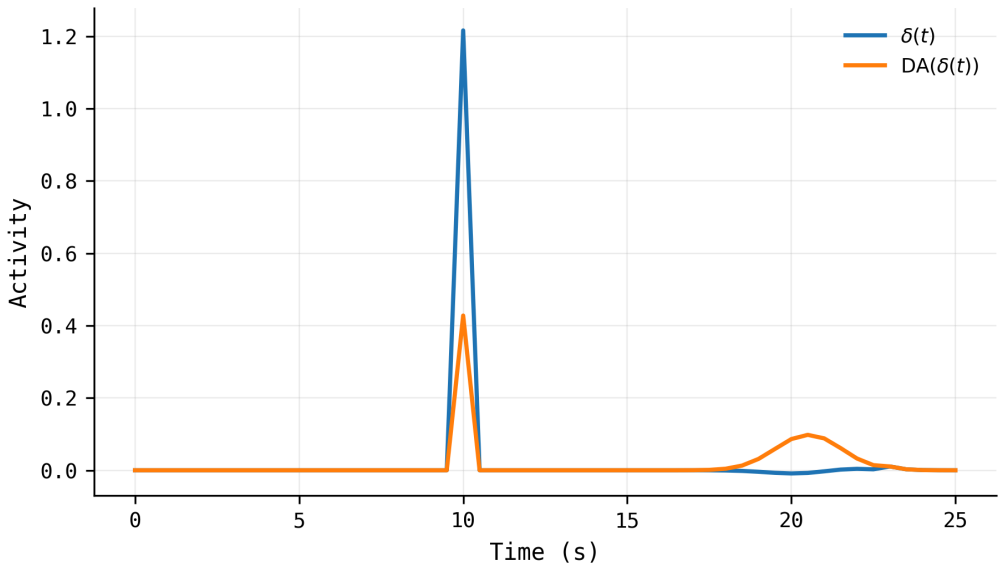

Figure 5: Average $\delta(t)$ compared with the dopamine signal $\mathrm{DA}(\delta(t))$ after applying the asymmetric nonlinearity.

(e) The DA nonlinearity compresses both negative values (by $\alpha = 6$) and large positive values (by $\beta = 6$ above $x^* = 0.27$). As a result, the negative dip at reward omission time is much attenuated in $\mathrm{DA}(\delta)$ compared to δ itself, while the positive peaks are also somewhat compressed. This matches the limited dynamic range of real dopamine neurons: they cannot fire below zero (floor effect) and saturate at high firing rates. The average DA signal therefore appears more asymmetric than the raw TD error, with a larger positive-to-negative ratio.

Question 6

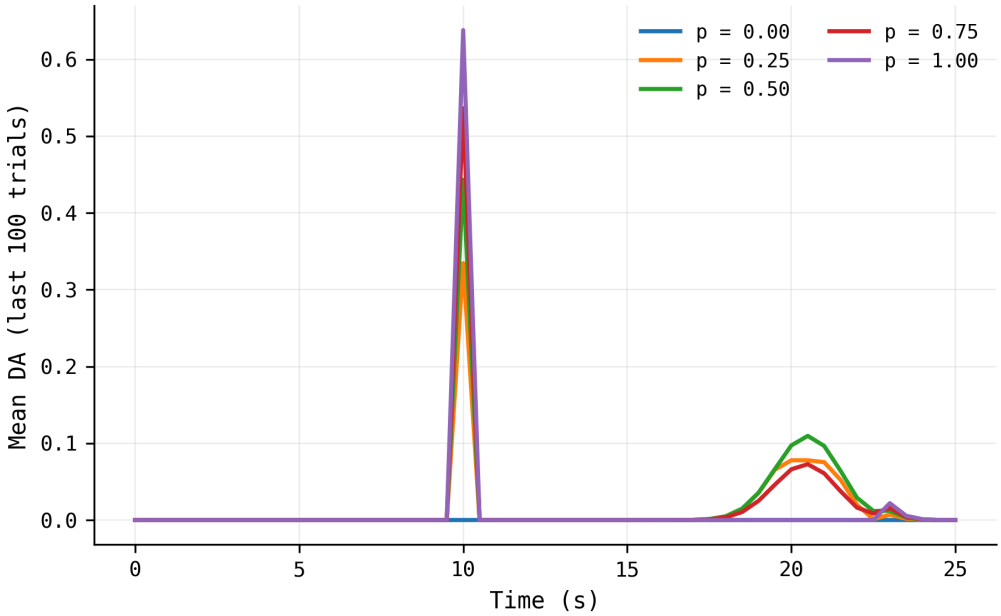

Figure 6: Average dopamine time course (last 100 trials) for each reward probability $p \in \{0, 0.25, 0.5, 0.75, 1\}$.

(a) At $p = 0$ there is no reward and hence no learning: the DA signal is flat at zero. As p increases, a stimulus-onset peak develops and grows — higher p means the stimulus is a stronger predictor of reward. The reward-time response is non-monotonic: it is absent at $p = 0$ and $p = 1$ (no surprise), and largest at intermediate p where reward delivery is most uncertain. At $p = 1$ the stimulus-onset peak is maximal and the reward-time response vanishes entirely because reward is fully predicted.

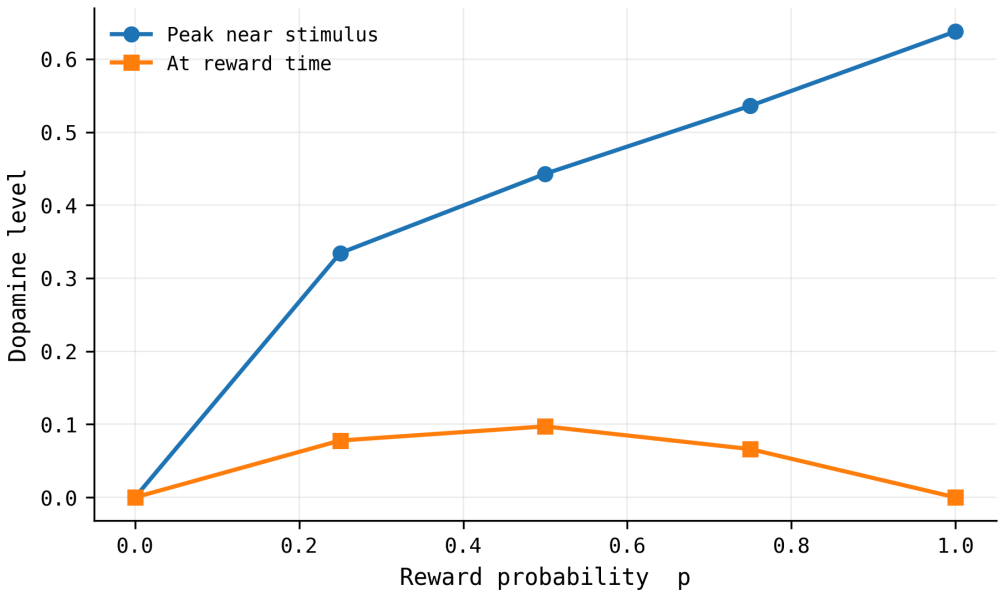

Figure 7: Dopamine level at the stimulus peak and at reward time as a function of reward probability p.

(b) The stimulus-time DA peak increases monotonically with p: it scales with the learned value of the stimulus, which is proportional to the expected reward $p \cdot r$. The reward-time DA peak is *non-monotonic* and resembles an inverted-U. At $p = 0$ and $p = 1$ reward-time DA is near zero (no surprise); it peaks at intermediate p because the mismatch between actual reward (when delivered) and expected reward $p \cdot r$ is largest there. More precisely, reward-time δ on rewarded trials is $r - p \cdot r = (1-p)r$, but these occur only a fraction p of the time, so the *average* reward-time DA reflects a mixture of positive and negative δ values weighted by p and $1-p$ respectively, passed through the compressive nonlinearity.

(c) If dopamine encoded *uncertainty* rather than prediction error, we would expect the stimulus-time DA peak to follow an inverted-U shape (maximal at $p = 0.5$, where outcome uncertainty $p(1-p)$ is highest). Instead, Figure 7 shows that the stimulus peak increases monotonically with p, inconsistent with an uncertainty code. The reward-time inverted-U could superficially resemble uncertainty, but it arises naturally from the asymmetric averaging of positive and negative prediction errors through the DA nonlinearity, not from uncertainty encoding per se.

Question 7

(a) For a $d \times d$ visual stimulus, the tapped delay line features are $\varphi_{mn\tau}(\boldsymbol{S}) = S_{mn\tau} = y_{mn}(t-\tau)$. The boxcar representation sums over the time lag:

$$\varphi^{\square}_{mn\tau}(\boldsymbol{S}) = \sum_{u=0}^{\tau} S_{mnu} = \sum_{u=0}^{\tau} y_{mn}(t-u) \tag{4}$$

Each boxcar feature detects the total presence of pixel (m, n) over the most recent $\tau + 1$ time steps, providing temporal integration of visual input.

(b.i) Each feature detector $\varphi^*_{i\tau}(\boldsymbol{S}; \boldsymbol{v}) = \sum_{u=0}^{\tau} h\left(\sum_{m,n=1}^{d} v^{(i)}_{mn} S_{mnu}\right)$ first computes a weighted sum of the stimulus frame at lag u, using spatial weights $v^{(i)}_{mn}$, then applies a nonlinearity $h(\cdot)$. The weights $\left\{v^{(i)}_{mn}\right\}^{d}_{m,n=1}$ define a spatial receptive field for feature i — they select which pixels (and with what polarity/gain) contribute to the feature's activation. This is analogous to a convolutional filter. The summation over lags $u = 0, ..., \tau$ integrates the filtered response over time, and $h(\cdot)$ introduces a nonlinear detection threshold.

(b.ii) With $a_{iu} = \sum_{m,n=1}^{d} v^{(i)}_{mn} S_{mnu}$ and $\varphi^*_{i\tau} = \sum_{u=0}^{\tau} h(a_{iu})$, the value function is $f(\boldsymbol{S}, \boldsymbol{\theta}) = \sum_{i\tau} w_{i\tau} \varphi^*_{i\tau}$. The update for $w_{i\tau}$ follows Equation 1 directly:

$$w_{i\tau} \leftarrow w_{i\tau} + \varepsilon \delta_t \varphi^*_{i\tau}(\boldsymbol{S}; \boldsymbol{v}) \tag{5}$$

For the spatial weights, applying the chain rule:

$$\frac{\partial f}{\partial v_{mn}^{(i)}} = \sum_{\tau} w_{i\tau} \sum_{u=0}^{\tau} h'(a_{iu}) S_{mnu} \tag{6}$$

so the update is:

$$v_{mn}^{(i)} \leftarrow v_{mn}^{(i)} + \varepsilon \delta_t \sum_{\tau} w_{i\tau} \sum_{u=0}^{\tau} h'(a_{iu}) S_{mnu} \tag{7}$$

The w update is identical in form to Q2 (linear in features). The $\boldsymbol{v}$ update is different: it involves the derivative h' and couples across time lags, making it a nonlinear gradient descent step that adapts the receptive fields.

(b.iii) With $h(x) = x$, we have $h'(x) = 1$ and $\varphi_{i\tau}^{*} = \sum_{u=0}^{\tau} a_{iu} = \sum_{m,n} v_{mn}^{(i)} \varphi_{mn\tau}^{\square}$. The feature detectors become *linear* combinations of boxcar features weighted by spatial filters. The $\boldsymbol{v}$ update simplifies to:

$$v_{mn}^{(i)} \leftarrow v_{mn}^{(i)} + \varepsilon \delta_t \sum_{\tau} w_{i\tau} \varphi_{mn\tau}^{\square} \tag{8}$$

Advantage: the model is jointly linear in $\boldsymbol{w}$ and $\boldsymbol{v}$ (though their product is bilinear), keeping gradients simple. Disadvantage: without a nonlinearity, different features φ_i^{*} cannot capture non-overlapping stimulus patterns — two features with spatial weights $\boldsymbol{v}^{(i)}$ and $\boldsymbol{v}^{(j)}$ that are both active for the same stimulus cannot be selectively gated, limiting representational power.

(b.iv) In the TD framework, the spatial weights $\boldsymbol{v}^{(i)}$ adapt to maximise reward prediction. In the Henschke et al. experiment, the reward-associated grating (group iii) is paired with water reward, so TD learning drives $\boldsymbol{v}^{(i)}$ to better encode that orientation, increasing the corresponding feature response. For groups without reward (i, ii), there is no prediction error to drive $\boldsymbol{v}$ updates, so V1 selectivity is unchanged. The model predicts that reward-paired stimuli sharpen the spatial filters that detect them, matching the elevated orientation selectivity observed in the goal-directed VR group.

Question 8

(a) In the standard linear TD model, the TD error $\delta(t) = r(t - \Delta t) + \Delta \hat{V}(t)$ is a *scalar* shared across all features. Each weight update $w_k \leftarrow w_k + \varepsilon \delta_t \varphi_k(s_t)$ depends on δ and the feature's own activation. A dopamine neuron that receives input from a specific feature k would modulate its response by φ_k, naturally producing feature-specific apparent dopamine responses even though δ itself is global. So standard TD can account for feature-specific responses if different dopamine neurons receive input from different features, without requiring multiple independent TD errors.

(b) Define the feature-specific value component $\hat{V}_k^{\pi}(s) = w_k \varphi_k(s)$ so that $\hat{V}^{\pi}(s) = \sum_k \hat{V}_k^{\pi}(s)$. A natural feature-specific TD error is:

$$\delta_k(t) = r_k(t - \Delta t) + \gamma \hat{V}_k^{\pi}(s_{t+1}) - \hat{V}_k^{\pi}(s_t) \tag{9}$$

where r_k is the portion of reward attributed to feature k. If we set $r_k = r \cdot \varphi_k(s_t) / \left(\sum_j \varphi_j(s_t) \right)$ (reward allocated proportionally to active features), then summing over k recovers the global TD error: $\sum_k \delta_k = \delta$. The weight update $w_k \leftarrow w_k + \varepsilon \delta_k \varphi_k$ produces equivalent learning because each δ_k drives only its own weight. This model is backward compatible: globally, $\sum_k \delta_k$ reproduces all the standard dopamine phenomena (shift from reward to stimulus, partial reinforcement effects, inverted-U). Locally, each δ_k is modulated by feature k's activation, producing the feature-specific specialisation observed by Engelhard et al. (2019): dopamine neurons with different feature inputs would show different response profiles while all contributing to a consistent global prediction error.

Appendix A: Code

Scratchpad for TD algorithm:

```
import random

R = [0.0, 0.0, 0.0, 0.0, 1.0]
V = [0.0, 0.0, 0.0, 0.0, 0.0]
gamma = 0.9
alpha = 0.1
max_episodes = 1000
max_steps = 20

for episode in range(max_episodes):
    state = random.randint(0, 4)  # start state
    for step in range(max_steps):
      if random.randint(0, 1) == 0:
        next_state = min(state + 1, 4)  # move right, cap at S4
      else:
        next_state = max(state - 1, 0)  # move left, cap at S0

      reward = R[next_state]
      V[state] += alpha * (reward + gamma * V[next_state] - V[state])
      state = next_state
      if state == 4:  # episode ends when reaching S4
        break

print("Estimated Values:", V)
```

Main experiment code:

```
import numpy as np
from dataclasses import dataclass
import shutil
from pathlib import Path

import matplotlib as mpl
import matplotlib.pyplot as plt

FIG_DIR = Path("src/figures")


@dataclass
class Parameters:
  # from the assignment
  t_max: float = 25.0
  dt: float = 0.5
  stimulus_time: float = 10.0
  reward_center: float = 20.0
  reward_sigma: float = 1.0
  reward_scale: float = 0.5
  gamma: float = 1.0
  lr: float = 0.2
  memory_span: float = 12.0
  n_trials: int = 201
  representation: str = "tapped"       # "tapped" or "boxcar"
  reward_probability: float = 1.0      # fraction of trials with reward (Q5, Q6)
  seed: int = 42                       # RNG seed for partial reinforcement

  @property
  def n_mem(self) -> int:
    return int(round(self.memory_span / self.dt)) + 1

@dataclass
class State:
  t: "np.ndarray"
  stim: "np.ndarray"
  reward: "np.ndarray"
  V_storage: "np.ndarray"
  dV_storage: "np.ndarray"
  error_storage: "np.ndarray"
  state_matrix: "np.ndarray"
  phi: "np.ndarray"
  w: "np.ndarray"
```

```python
    rewarded: "np.ndarray | None" = None  # bool array tracking which trials had reward


def initialize_stimulus(t: np.ndarray, t_stim: float, dt: float) -> np.ndarray:
    """
    Create a stimulus signal that is zero everywhere except for a single spike of
    amplitude 1.0 at time t_stim. The stimulus is represented as a 1-D array
    aligned with the time grid t, which is generated using the specified time step dt.
    """
    stim = np.zeros_like(t)
    idx = int(round(t_stim / dt))
    stim[idx] = 1.0
    return stim


def initialize_reward(t: np.ndarray, t_reward: float, sigma: float) -> np.ndarray:
    """
    Create a reward signal that follows a Gaussian distribution centered at
    t_reward with standard deviation sigma. The reward is represented as a
    1-D array aligned with the time grid t.
    """
    return np.exp(-((t - t_reward) ** 2) / (2.0 * sigma**2))


def intitialize_state_matrix(stim: np.ndarray, n_mem: int) -> np.ndarray:
    """Build the stimulus-by-time-delay state matrix S for all time steps.

    Constructs the matrix S used to represent the recent history of the stimulus
    signal. Each row corresponds to an absolute time step t, and each column
    corresponds to a relative delay tau (how many steps ago). The entry S[t, tau]
    records the stimulus level at time (t - tau), i.e. what the stimulus was tau
    steps in the past when viewed from time t.

    For the tapped delay line representation, each entry of S is used
    directly as a feature: phi_{i,tau}(S) = S_{i,tau}. For the boxcar
    representation (Q4), cumulative sums over the tau axis are taken
    instead.

    Boundary handling: at early time steps where t < tau, the stimulus
    history is unavailable (we cannot look before t = 0), so those
    entries remain zero. This is enforced by the inner loop bound
    min(ti, n_mem - 1).

    Args:
        stim: 1-D array of stimulus values at each time step, shape
              (steps,). For Q3, this is a single spike (1.0 at the
              stimulus time, 0.0 elsewhere).
        n_mem: Number of memory slots (columns), equal to
               int(T_mem / dt) + 1. With T_mem = 12 s and dt = 0.5 s
               this gives n_mem = 25, so S is a 1 x 25 matrix per
               time step (for a single stimulus).

    Returns:
        2-D array of shape (steps, n_mem) where steps = len(stim).
        Entry S[t, tau] = stim[t - tau] for tau <= t, and 0 otherwise.
    """
    steps = stim.shape[0]
    S = np.zeros((steps, n_mem), dtype=float)
    for ti in range(steps):
      for tau in range(min(ti, n_mem - 1) + 1):
        S[ti, tau] = stim[ti - tau]
    return S


def initialize_state(params: Parameters) -> State:
    t = np.arange(0, params.t_max + params.dt, params.dt)
    steps = t.shape[0]
    V_storage = np.zeros((params.n_trials, steps), dtype=float)
    dV_storage = np.zeros((params.n_trials, steps), dtype=float)
    error_storage = np.zeros((params.n_trials, steps), dtype=float)
    stim = initialize_stimulus(t, params.stimulus_time, params.dt)
    reward = params.reward_scale * initialize_reward(t, params.reward_center, params.reward_sigma)
    state_matrix = intitialize_state_matrix(stim, params.n_mem)

    # Select feature function based on representation type
```

```python
  if params.representation == "tapped":
    feat_fn = tapped_features
  elif params.representation == "boxcar":
    feat_fn = boxcar_features
  else:
    raise ValueError(f"Unknown representation: {params.representation}")

  phi = np.zeros((steps, params.n_mem), dtype=float)
  for ti in range(steps):
    phi[ti] = feat_fn(state_matrix[ti])

  w = np.zeros(params.n_mem, dtype=float)

  return State(
    t=t,
    stim=stim,
    reward=reward,
    state_matrix=state_matrix,
    V_storage=V_storage,
    dV_storage=dV_storage,
    error_storage=error_storage,
    phi=phi,
    w=w,
    rewarded=np.zeros(params.n_trials, dtype=bool),
  )


def tapped_features(state_row: np.ndarray) -> np.ndarray:
  """Tapped delay line feature detector: phi_{i,tau}(S) = S_{i,tau}.

  Each feature simply returns the stimulus level at a specific delay.
  This is the identity mapping over the state row — the simplest
  possible feature representation for the state history.

  Args:
      state_row: A single row of the state matrix S, shape (n_mem,).

  Returns:
      A copy of the state row (copy to avoid aliasing issues).
  """
  return state_row.copy()


def boxcar_features(state_row: np.ndarray) -> np.ndarray:
  """Boxcar feature detector: phi^box_{i,tau}(S) = sum_{u=0}^{tau} S_{i,u}.

  Each feature returns the cumulative presence of the stimulus over
  a history window of length tau. This allows generalisation across
  nearby time delays.

  Args:
      state_row: A single row of the state matrix S, shape (n_mem,).

  Returns:
      Cumulative sum along the delay axis.
  """
  return np.cumsum(state_row)


def run_td(params: Parameters | None = None) -> State:
  """Run TD learning with the given parameters.

  Supports tapped delay line (Q3) and boxcar (Q4) representations,
  deterministic reward (Q3, Q4) and partial reinforcement (Q5, Q6).

  Args:
      params: Simulation parameters. Uses Q3 defaults if None.

  Returns:
      State object with V_storage, dV_storage, error_storage populated.
  """
  if params is None:
    params = Parameters()

  rng = np.random.default_rng(params.seed)
```

```python
    s = initialize_state(params)
    steps = s.t.shape[0]

    for trial in range(params.n_trials):
      # Partial reinforcement: reward present with probability p
      is_rewarded = rng.random() < params.reward_probability
      s.rewarded[trial] = is_rewarded
      reward = s.reward if is_rewarded else np.zeros_like(s.reward)

      v = np.zeros(steps, dtype=float)
      dv = np.zeros(steps, dtype=float)
      delta = np.zeros(steps, dtype=float)
      v[0] = float(s.w @ s.phi[0])

      for ti in range(1, steps):
        v_curr = float(s.w @ s.phi[ti])
        dv[ti] = params.gamma * v_curr - v[ti - 1]
        delta[ti] = reward[ti - 1] + dv[ti]
        s.w += params.lr * delta[ti] * s.phi[ti - 1]
        v[ti] = float(s.w @ s.phi[ti])

      s.V_storage[trial] = v
      s.dV_storage[trial] = dv
      s.error_storage[trial] = delta

    return s

def _setup_style() -> None:
  mpl.rcParams.update({
    "font.family": "monospace",
    "font.size": 10,
    "axes.titlesize": 11,
    "axes.labelsize": 10,
    "xtick.labelsize": 9,
    "ytick.labelsize": 9,
    "legend.fontsize": 8.5,
    "figure.dpi": 150,
    "savefig.dpi": 300,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.linewidth": 0.8,
    "xtick.major.width": 0.8,
    "ytick.major.width": 0.8,
    "lines.linewidth": 1.4,
    "grid.alpha": 0.25,
    "grid.linewidth": 0.5,
  })


def _save(fig: plt.Figure, subdir: str, name: str) -> None:
  out = FIG_DIR / subdir
  out.mkdir(parents=True, exist_ok=True)
  fig.savefig(out / name, dpi=300, bbox_inches="tight")
  plt.close(fig)


def plot_stim_reward(s: State, subdir: str = "question-3", name: str = "q3a_stim_reward.png") -> None:
  """Plot stimulus and reward as functions of time."""
  fig, axes = plt.subplots(2, 1, sharex=True, figsize=(7, 3.6),
                           gridspec_kw={"hspace": 0.12})

  axes[0].plot(s.t, s.stim, lw=1.6)
  axes[0].set_ylabel("Stimulus  y(t)")
  axes[0].set_ylim(-0.05, 1.15)
  axes[0].set_yticks([0, 1])
  axes[0].grid(True)

  axes[1].plot(s.t, s.reward, lw=1.6)
  axes[1].set_ylabel("Reward  r(t)")
  axes[1].set_xlabel("Time (s)")
  axes[1].grid(True)

  fig.align_ylabels(axes)
  _save(fig, subdir, name)
```

```python
def plot_learning_curves(s: State, n_trials: int,
                         subdir: str = "question-3",
                         name: str = "q3b_td_learning.png") -> None:
  """Plot V̂(t), ΔV̂(t), δ(t) for every 10th trial."""
  trial_indices = list(range(0, n_trials, 10))
  n_curves = len(trial_indices)

  cmap = mpl.colormaps["viridis_r"]
  colours = [cmap(i / max(n_curves - 1, 1)) for i in range(n_curves)]

  labels = [r"$\hat{V}(t)$", r"$\Delta\hat{V}(t)$", r"$\delta(t)$"]
  keys = ["V_storage", "dV_storage", "error_storage"]

  fig, axes = plt.subplots(3, 1, sharex=True, figsize=(7, 7.5),
                           gridspec_kw={"hspace": 0.10})

  for ax, key, label in zip(axes, keys, labels):
    data = getattr(s, key)
    for ci, tr in enumerate(trial_indices):
      ax.plot(s.t, data[tr], color=colours[ci], lw=1.0, alpha=0.9)
    ax.set_ylabel(label)
    ax.grid(True)

  axes[-1].set_xlabel("Time (s)")

  sm = mpl.cm.ScalarMappable(
    cmap="viridis_r",
    norm=mpl.colors.Normalize(vmin=1, vmax=n_trials),
  )
  sm.set_array([])
  cbar = fig.colorbar(sm, ax=axes, location="right", fraction=0.025, pad=0.02)
  cbar.set_label("Trial", fontfamily="monospace")

  fig.align_ylabels(axes)
  _save(fig, subdir, name)


def dopamine_nonlinearity(x: np.ndarray, alpha: float = 6.0, beta: float = 6.0,
                          x_star: float = 0.27) -> np.ndarray:
  """Piecewise-linear dopamine activation DA(x) from Q5(e).

  Compresses negative values by 1/alpha and saturates large positives
  above x* with slope 1/beta.
  """
  y = np.empty_like(x)
  neg = x < 0.0
  mid = (x >= 0.0) & (x < x_star)
  hi = x >= x_star
  y[neg] = x[neg] / alpha
  y[mid] = x[mid]
  y[hi] = x_star + (x[hi] - x_star) / beta
  return y


def plot_last100_by_type(s: State, subdir: str, name: str,
                         n_last: int = 100) -> None:
  """Q5(a): average V̂, ΔV̂, δ over last n_last trials, split by rewarded/unrewarded/all."""
  t = s.t
  r_mask = s.rewarded[-n_last:]

  labels_keys = [
    (r"$\hat{V}(t)$", "V_storage"),
    (r"$\Delta\hat{V}(t)$", "dV_storage"),
    (r"$\delta(t)$", "error_storage"),
  ]

  fig, axes = plt.subplots(3, 1, sharex=True, figsize=(7, 7.5),
                           gridspec_kw={"hspace": 0.10})

  for ax, (label, key) in zip(axes, labels_keys):
    data = getattr(s, key)[-n_last:]
    ax.plot(t, data.mean(axis=0), color="black", lw=1.8, label="all")
    if r_mask.any():
      ax.plot(t, data[r_mask].mean(axis=0), color="tab:green",
```

```
            lw=1.4, ls="--", label="rewarded")
    if (~r_mask).any():
      ax.plot(t, data[~r_mask].mean(axis=0), color="tab:red",
            lw=1.4, ls="--", label="unrewarded")
    ax.set_ylabel(label)
    ax.grid(True)

  axes[0].legend(frameon=False, ncol=3)
  axes[-1].set_xlabel("Time (s)")
  fig.align_ylabels(axes)
  _save(fig, subdir, name)


def plot_da_compare(s: State, subdir: str, name: str,
                    n_last: int = 100) -> None:
  """Q5(e): average δ vs DA(δ) over last n_last trials."""
  t = s.t
  delta_tail = s.error_storage[-n_last:]
  da_tail = dopamine_nonlinearity(delta_tail)

  fig, ax = plt.subplots(1, 1, figsize=(7, 3.8))
  ax.plot(t, delta_tail.mean(axis=0), lw=1.8, label=r"$\delta(t)$")
  ax.plot(t, da_tail.mean(axis=0), lw=1.8, label=r"$\mathrm{DA}(\delta(t))$")
  ax.set_xlabel("Time (s)")
  ax.set_ylabel("Activity")
  ax.grid(True)
  ax.legend(frameon=False)
  _save(fig, subdir, name)


def plot_da_sweep(t: np.ndarray, da_curves: dict[float, np.ndarray],
                  subdir: str, name: str) -> None:
  """Q6(a): overlay average DA time courses for different reward probabilities."""
  fig, ax = plt.subplots(1, 1, figsize=(7, 4.2))
  for p in sorted(da_curves):
    ax.plot(t, da_curves[p], lw=1.8, label=f"p = {p:.2f}")
  ax.set_xlabel("Time (s)")
  ax.set_ylabel("Mean DA (last 100 trials)")
  ax.grid(True)
  ax.legend(frameon=False, ncol=2)
  _save(fig, subdir, name)


def plot_da_peaks(ps: np.ndarray, peak_stim: np.ndarray,
                  at_reward: np.ndarray, subdir: str, name: str) -> None:
  """Q6(b): DA level at stimulus peak and at reward time vs p."""
  fig, ax = plt.subplots(1, 1, figsize=(7, 4))
  ax.plot(ps, peak_stim, marker="o", lw=1.8, label="Peak near stimulus")
  ax.plot(ps, at_reward, marker="s", lw=1.8, label="At reward time")
  ax.set_xlabel("Reward probability  p")
  ax.set_ylabel("Dopamine level")
  ax.grid(True)
  ax.legend(frameon=False)
  _save(fig, subdir, name)


def main():
  _setup_style()

  if FIG_DIR.exists():
    shutil.rmtree(FIG_DIR)
  FIG_DIR.mkdir(parents=True)

  # Q3: tapped delay line, eps=0.2, N=201
  q3_params = Parameters(representation="tapped", lr=0.2, n_trials=201)
  state_q3 = run_td(q3_params)
  plot_stim_reward(state_q3, subdir="question-3", name="q3a_stim_reward.png")
  plot_learning_curves(state_q3, n_trials=q3_params.n_trials,
                       subdir="question-3", name="q3b_td_learning.png")

  # Q4: boxcar, eps=0.01, N=201
  q4_params = Parameters(representation="boxcar", lr=0.01, n_trials=201)
  state_q4 = run_td(q4_params)
  plot_learning_curves(state_q4, n_trials=q4_params.n_trials,
                       subdir="question-4", name="q4a_boxcar_learning.png")
```

```python
  # Q5: partial reinforcement, boxcar, eps=0.01, p=0.5, N=1000
  q5_params = Parameters(representation="boxcar", lr=0.01, n_trials=1000,
                         reward_probability=0.5, seed=11)
  state_q5 = run_td(q5_params)
  plot_last100_by_type(state_q5, subdir="question-5", name="q5a_last100_by_type.png")
  plot_da_compare(state_q5, subdir="question-5", name="q5e_da_nonlinearity.png")

  # Q6: reward probability sweep
  dt = 0.5
  t_stim = 10.0
  t_reward = 20.0
  p_values = np.array([0.0, 0.25, 0.5, 0.75, 1.0])
  stim_idx = int(round(t_stim / dt))
  reward_idx = int(round(t_reward / dt))
  window = 2  # +/- 1 s around stimulus for peak detection

  da_curves: dict[float, np.ndarray] = {}
  peak_stim = np.zeros_like(p_values)
  at_reward = np.zeros_like(p_values)
  t_grid = None

  for i, p in enumerate(p_values):
    q6_params = Parameters(
      representation="boxcar", lr=0.01, n_trials=1000,
      reward_probability=float(p), seed=31 + i,
    )
    state_q6 = run_td(q6_params)
    if t_grid is None:
      t_grid = state_q6.t

    # Apply DA nonlinearity per-trial, then average over last 100
    da = dopamine_nonlinearity(state_q6.error_storage[-100:]).mean(axis=0)
    da_curves[float(p)] = da

    lo = max(0, stim_idx - window)
    hi = min(da.shape[0], stim_idx + window + 1)
    peak_stim[i] = np.max(da[lo:hi])
    at_reward[i] = da[reward_idx]

  plot_da_sweep(t_grid, da_curves, subdir="question-6", name="q6a_da_sweep.png")
  plot_da_peaks(p_values, peak_stim, at_reward,
                subdir="question-6", name="q6b_da_peaks.png")

  print(f"Figures saved to {FIG_DIR.resolve()}")



if __name__ == "__main__":
  main()
```

Appendix B: AI Prompts

GitHub Copilot — autocomplete during report writing, primarily for equation typesetting.

Claude Code — the following prompts were used while coding and drafting the report:

- `read src/coursework/assignment.pdf and give me a tip for question 1`
- `give a hint for question 2`
- `walk me through example of value functions from reinforcement learning`
- `how do we go from this to TD learning?`
- `explain question 3`
- `in src/lib/assignment.py check the indexing for the sliding window in state S`
- `src/lib/assignment.py defines an experiment in code, add the plots needed by src/pdfs/assignment.pdf question 3 as publication quality matplotlib plots with monospace font, and save the plots to src/report/figures/question-3`
- `refine the report so far`
- `read the assignment and report and bugs in the code`
- `in src/lib/assignment.py properly parameterise run_td_tapped to support both tapped and boxcar representations.`
- `add the plots for question-4 in the usual way to src/lib/assignment.py`
- `now add in question-5, same as question-4 except reward\_probability=0.5, n\_trials=1000 and seed=11`
- `now for question-6 do a sweep of the reward probabilities of [0.0,0.25,0.5,0.75,1.0] and make the appropriate plots`
- `give a hint for question 7`
- `give a hint for question 8`
- `refine the report so far` (ran multiple times)
- `refine the report and code for publication`